



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

ОТЧЕТ

по лабораторной работе №2

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация алгоритма геопоиска (kd-tree)»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	kd-tree	3
1.2	Сложность операций	3
2	Практическая часть	4
2.1	Реализация структуры	4
2.2	Реализация бенчмарка	4
3	Исследовательская часть	5
3.1	Аппаратные характеристики	5
3.2	Методика замеров	5
3.3	Результаты бенчмарка	6
4	Вывод	9

1 Теоретическая часть

1.1 kd-tree

kd-tree (*k-dimensional tree*) — это двоичное дерево поиска для точек в k -мерном пространстве. В данной работе используется случай $k = 2$ с координатами (x, y) . На каждом уровне дерева выбирается ось разбиения (чередование $x, y, x, y, \dots$), а в узел помещается точка-медиана по текущей оси.

Базовые операции:

- **insert**: спуск по дереву с чередованием осей и добавление нового листа;
- **find_nearest**: поиск ближайшей точки к заданной.

Для оценки расстояния используется евклидова метрика (квадрат расстояния, без извлечения корня):

$$d^2((x_1, y_1), (x_2, y_2)) = (x_1 - x_2)^2 + (y_1 - y_2)^2.$$

1.2 Сложность операций

В среднем для сбалансированного дерева:

- вставка: $O(\log n)$;
- поиск ближайшего соседа: около $O(\log n)$, но в худшем случае может деградировать до $O(n)$.

В исследовании отдельно измеряются две операции (**insert** и **find_nearest**) и для каждой фиксируются время на операцию и пропускная способность с 95% доверительным интервалом.

2 Практическая часть

2.1 Реализация структуры

Публичный интерфейс `kd-tree` реализован в `inc/kdtree.h`:

- `new_tree_from_csv` — построение дерева по CSV с координатами x, y ;
- `insert` — добавление узла с точкой;
- `find_nearest` — поиск ближайшей точки к заданной (x, y) ;
- `free_tree` — освобождение памяти дерева.

Для построения дерева входные точки читаются в массив пар координат, после чего дерево рекурсивно строится по медиане текущего подмассива с чередованием осей разбиения.

2.2 Реализация бенчмарка

Бенчмарк реализован в файле `src/benchmark.c`. Генерация входных датасетов выполняется скриптом `scripts/generate_bench_data.py`.

Выходной CSV для каждого датасета содержит:

- среднее время вставки на операцию и 95% ДИ;
- среднюю пропускную способность вставки (ops/sec) и 95% ДИ;
- среднее время поиска на операцию и 95% ДИ;
- среднюю пропускную способность поиска (ops/sec) и 95% ДИ.

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS 26.3.1 (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- компилятор C: gcc, стандарт C17.

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания для обеспечения максимальной производительности.

3.2 Методика замеров

Для каждого файла из `data/bench/manifest.txt` выполняются следующие шаги:

- 1) загрузка точек из CSV;
- 2) серия замеров `insert` (вставляются случайные точки в диапазоне координат датасета);
- 3) серия замеров `find_nearest` (поиск ближайшей точки для координат из датасета).

Параметры замеров:

- `BENCH_ITEERS=100` итераций (количество итераций);
- 100 операций на итерацию для каждой операции.

95% доверительный интервал рассчитывается по формуле:

$$\Delta_{95} = 1.96 \cdot \sqrt{\frac{s^2}{n}},$$

где s^2 — выборочная дисперсия по итерациям, n — число итераций.

3.3 Результаты бенчмарка

В таблице 3.1 приведена задержка операций, в таблице 3.2 — пропускная способность.

Таблица 3.1 – Задержка операций kd-tree (среднее по итерациям)

N	вставка, нс/оп	поиск, нс/оп
5000	122.7 ± 8.3	100.5 ± 2.1
10000	120.8 ± 2.6	110.3 ± 3.3
20000	122.5 ± 1.8	111.6 ± 2.5
35000	128.6 ± 1.0	128.0 ± 3.5
50000	138.3 ± 2.4	132.7 ± 2.5
75000	146.7 ± 2.1	152.8 ± 2.4
100000	165.2 ± 7.6	154.3 ± 3.0
250000	352.0 ± 17.5	202.7 ± 7.3
500000	795.2 ± 29.2	229.3 ± 20.9
750000	1093.2 ± 43.9	245.6 ± 29.6
1000000	1306.7 ± 53.5	247.9 ± 31.8

Таблица 3.2 – Пропускная способность операций kd-tree (среднее по итерациям)

N	вставка, млн оп/с	поиск, млн оп/с
5000	8.61 ± 0.28	10.02 ± 0.14
10000	8.35 ± 0.14	9.19 ± 0.17
20000	8.20 ± 0.11	9.03 ± 0.13
35000	7.79 ± 0.06	7.91 ± 0.14
50000	7.28 ± 0.10	7.58 ± 0.09
75000	6.84 ± 0.08	6.57 ± 0.07
100000	6.26 ± 0.18	6.52 ± 0.09
250000	2.98 ± 0.12	5.00 ± 0.08
500000	1.30 ± 0.05	4.54 ± 0.08
750000	0.96 ± 0.04	4.30 ± 0.08
1000000	0.80 ± 0.04	4.28 ± 0.08

Из полученных результатов можно сделать следующие выводы:

- при малых и средних размерах данных вставка и поиск имеют сопоставимую задержку;
- при росте N задержка вставки растёт заметно быстрее, чем задержка поиска;
- пропускная способность поиска остаётся более стабильной на больших датасетах.

Графики результатов замеров представлены на рисунках ниже.

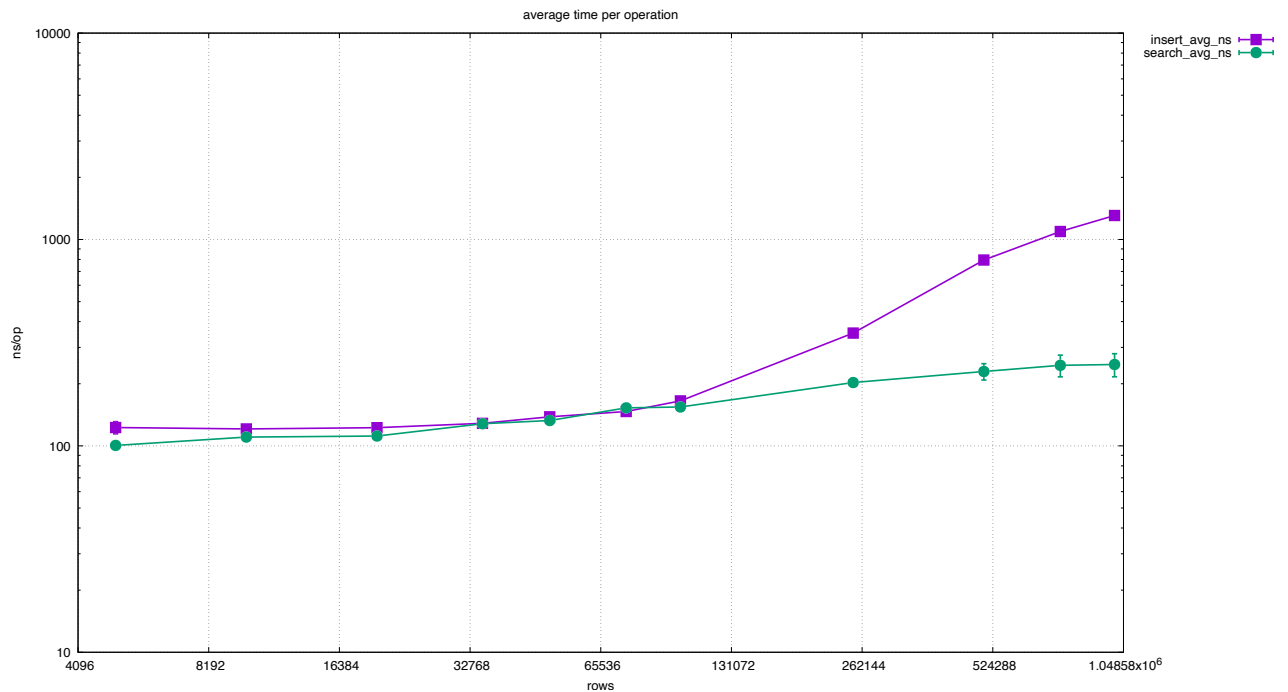


Рисунок 3.1 – Средняя задержка операций вставки и поиска для kd-tree

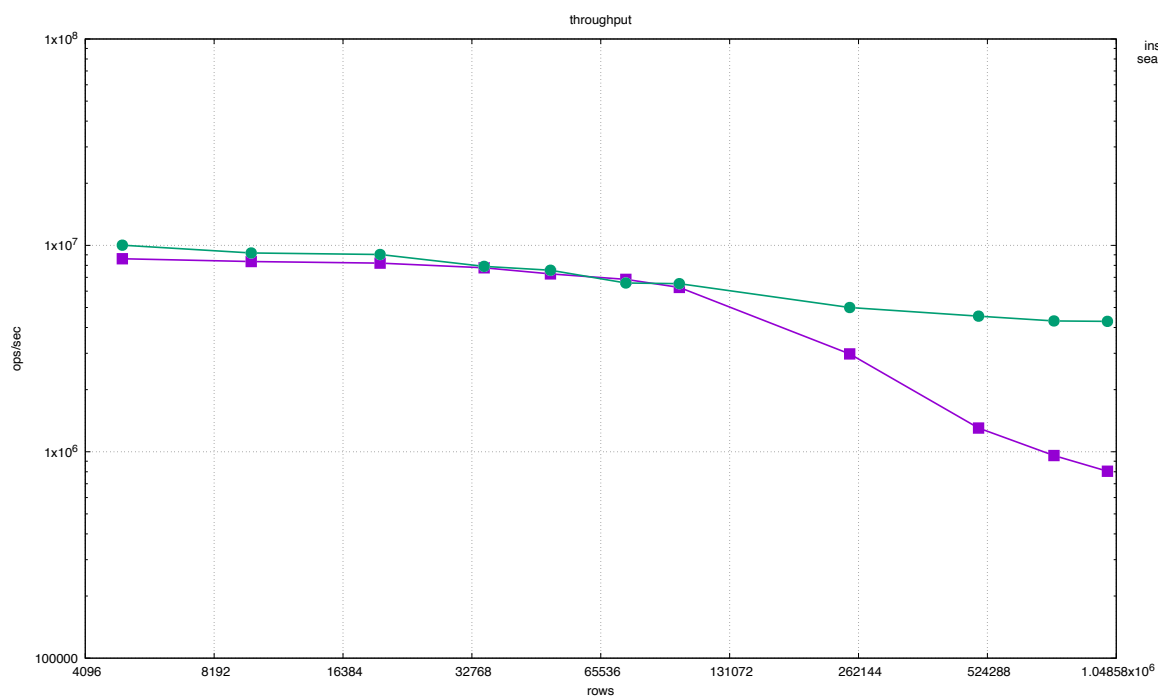


Рисунок 3.2 – Пропускная способность операций вставки и поиска для kd-tree

4 Вывод

В работе реализован kd-tree индекс для геопоиска с поддержкой построения из CSV, динамической вставки и поиска ближайшей точки. Подготовлен бенчмарк, который измеряет производительность операций `insert` и `find_nearest` на наборах данных различного размера и вычисляет 95% доверительные интервалы как для времени на операцию, так и для пропускной способности.

Экспериментально показано, что при росте размера данных операция поиска масштабируется стабильнее, чем операция вставки в динамически модифицируемое дерево. Полученные результаты оформлены в виде таблиц и графиков и могут использоваться для дальнейшей оптимизации реализации (например, улучшение стратегии балансировки и снижение накладных расходов на вставку).